

Experimental setup

Environment

Requirements: Python 3.11 or newer (enforced in `pyproject.toml`), plus an optional Rust toolchain (cargo, stable 1.70+) when building native acceleration crates under `../cogant/rust/`.

From the COGANT package root `../cogant/` (where `pyproject.toml` and `py/cogant/` live), install with `uv sync --all-extras`, or `pip install -e ".[dev,viz]"` / `pip install -e ".[all]"` as in `GETTING_STARTED.md`. Run those commands from that directory when working inside this monorepo layout. Python sources live under `../cogant/py/cogant/`; see the package `README.md`.

Running the API

Minimal **Session** run:

```
from cogant import Session

session = Session.from_target("./path/to/repo")
session.extract_static()
session.build_graph()
session.translate_to_gnn()
session.export_all("output/")
```

Minimal **Pipeline** run:

```
from cogant import PipelineRunner
from cogant.api.pipeline import PipelineConfig

runner = PipelineRunner()
config = PipelineConfig(output_dir="output/")
bundle = runner.run("./path/to/repo", config)
```

Adjust PipelineConfig.stages, skip_stages, and plugins to

Configuration files

YAML configuration can drive pipeline behavior (paths, stages, plugin options). The architecture and SPEC documents describe the configuration surface; keep project-specific secrets out of version control.

A minimal pipeline configuration looks like this:

```
# cogant.config.yaml
stages:
  - ingest
  - static
  - normalize
  - graph
  - dynamic           # optional; skipped if no coverage/trace
  - translate
  - statespace
  - process
  - export
  - validate
```

CLI

Use the cogant CLI for scripted batch runs—see
`../cogant/docs/CLI_GUIDE.md` for the command list that
matches the installed version.

Export targets

The primary export targets are the **Generalized Notation Notation (GNN)** canonical Markdown (`model.gnn.md`) and the equivalent companion JSON files described in `../cogant/docs/GNN_EXPORT.md`. Optional interop targets (GraphML, Parquet) support analysis in Gephi/yEd and DuckDB, and optional tensor views for PyTorch Geometric, DGL, or HDF5 can be selected when downstream graph neural network training pipelines need to consume the program graph as a relational tensor. Ensure the Python environment includes optional dependencies for these tensor exports when those code paths are used.

Python AST parser capabilities

The v0.1.x front end relies on `cogant.static.parser.PythonASTParser`, which processes Python source through the standard-library `ast` module at the CPython version available in the runtime (3.9+ recommended). The parser extracts the following construct categories:

- ▶ **Module-level entities:** module docstrings, `__all__` exports, top-level assignments.
- ▶ **Functions and methods:** `def` and `async def`, including signatures with positional, keyword, variadic (`*args`, `**kwargs`), and positional-only parameters. Default values are recorded as constant expressions where statically evaluable.
- ▶ **Classes:** class definitions, base classes, metaclasses, and the `__init__` / `__new__` boundary.
- ▶ **Decorators:** `@staticmethod`, `@classmethod`, `@property`, `@dataclass`, and arbitrary user-defined decorators. Decorator arguments are captured as attribute metadata.
- ▶ **Type annotations:** PEP 484 / 526 / 604 annotations on function parameters, return types, and variable assignments.

Progressive IR stages

Processing advances through six intermediate representations, each adding semantic detail atop its predecessor. Table 3 summarizes what each stage contributes.

Table 3. Progressive IR stages and their contributions.

Stage	IR name	Key additions	Typical output size (10K-function repo)
1	Repo IR	Raw entities and relationships per file; deduplication; merged type info	~15 MB JSON
2	Program Graph IR	Consolidated directed graph $G =$	~20 MB JSON

Performance characteristics

The architecture targets the following benchmarks on a 4-core machine, as specified in `../cogant/docs/ARCHITECTURE.md`:

Repository size	Target wall-clock time	Memory budget
10K functions	< 30 s	< 500 MB
100K functions	< 5 min	< 2 GB
1M functions	< 1 hr	< 2 GB (streaming)

These targets assume the Python orchestration layer with Rust acceleration on critical paths (graph construction, rule matching, and Generalized Notation Notation section/tensor packing in `cogant-gnn`). In the current v0.1.x release, where Rust bindings are staged rather than fully wired, Python fallback implementations handle graph operations; users should expect roughly $2\text{--}3\times$ slower throughput on repositories above 50K functions until native acceleration is enabled.

Parallelism is exploited at multiple stages: file parsing is

What to record

For reproducible experiments, record: COGANT version or commit hash, interpreter version, list of stages executed, configuration file contents (redacted), input repository commit hash, and random seeds for any learned components **outside** COGANT that consume the exports.